6. Constructive Logic: Heyting Semantics for Digital Composition

Constructive logic provides the semantic discipline for digital composition when one wants proofs that can be executed, transformed, and checked alongside the circuits they justify. In contrast to classical reasoning over Bool, constructive reasoning distinguishes between the existence of a value and the availability of a witness. For circuit design, this distinction is not merely philosophical: it determines whether a specification can be turned into a proof-carrying artifact whose correctness is preserved by construction.

A useful way to organize this section is through the error profile $\varepsilon(r)$, interpreted as a tradeoff between proof obligations and counterexample rate. Here r may be read as a resource parameter, refinement level, or search budget, depending on the verification workflow. As proof obligations increase, the counterexample rate should decrease; conversely, a low proof burden may leave more room for spurious implementations. The point of a constructive semantics is to make this tradeoff explicit and machine-checkable rather than implicit.

At the algebraic level, Heyting algebras generalize Boolean algebras by replacing excluded middle with an implication operation that is adjoint to conjunction. This matters for digital composition because many specifications are naturally intuitionistic: one can prove that a circuit satisfies a property without asserting that every proposition is decidable in advance. In a Boolean algebra, every element has a complement and every proposition is either true or false. In a Heyting algebra, negation is defined as implication into bottom, and double negation need not collapse to the original proposition. This asymmetry is precisely what makes constructive semantics sensitive to computational content.

The constructive XOR illustrates the difference. Classically, XOR is often treated as a derived Boolean connective with a simple truth table. Constructively, one must account for how the output is witnessed from the inputs. A constructive XOR specification can be expressed as a disjoint sum of cases, or as a proposition that carries enough information to extract a circuit without appealing to nonconstructive case splits. This is especially important when the target artifact is not just a truth-functional relation but an executable term.

The Curry–Howard correspondence supplies the bridge from logic to circuits: types as specifications, terms as implementations, and proofs as programs. Under this interpretation, a proposition about a digital component becomes a type, and a proof term inhabiting that type becomes a circuit or program satisfying the specification. The extraction pipeline then turns formal derivations into executable artifacts. In practice, this means that a proof in a dependently typed system can be compiled into a circuit description, preserving the logical structure of the specification in the generated implementation.

This perspective is particularly effective for closure questions. If a family of circuit specifications is closed under constructive connectives such as conjunction, disjunction, and implication, then one can build larger verified systems from smaller verified parts. Conjunction corresponds to pairing proofs and composing specifications in parallel; disjunction corresponds to tagged choice; implication corresponds to a transformation from one verified interface to another. These closure properties are the constructive analogue of algebraic closure under operations on Boolean functions, but now the operations carry proof terms as well as truth conditions.

However, double-negation caveats must be handled carefully. A statement of the form $\neg\neg P$ does not generally yield P constructively, so any workflow that relies on classical elimination principles must either justify them separately or restrict attention to fragments where they are admissible. For digital composition, this means that some seemingly natural equivalences are not available without additional assumptions, and the proof architecture must reflect that limitation. In particular, a proof that only establishes $\neg\neg$ of a specification is not yet a proof-carrying implementation.

A compact way to state the constructive discipline is to treat specifications as inhabited types and implementations as witnesses. Then the verification task is not merely to show that a relation holds, but to exhibit a term whose computational behavior realizes the relation. In a Heyting semantics, this aligns the algebra of propositions with the algebra of proof objects: conjunction corresponds to pairing, disjunction to tagged choice, and implication to function space. The resulting semantics is compositional in the strongest sense, because the proof structure itself mirrors the structure of the circuit or module being assembled.

The practical artifact associated with this section is a proof-carrying development in a system such as Agda or Coq, together with extracted circuits. In such a workflow, one encodes the specification of a gate, module, or composition law as a type; proves the desired properties constructively; and then extracts an implementation that is correct by construction relative to the proof. The resulting artifact is not merely a theorem statement but a reproducible build object: the proof script, the extracted term, and the corresponding circuit representation can all be checked independently.

Taken together, Heyting semantics, Curry–Howard, and proof extraction provide a disciplined foundation for digital composition. They clarify which logical principles are computationally meaningful, which equivalences are safe to use, and how to move from abstract specifications to verified implementations without leaving the constructive setting. In the broader development of this book, this section supplies the logical counterpart to the modular and traced composition principles developed earlier: where those sections organize how components connect, constructive logic governs what it means for a connection to be justified by a witness that can be extracted and reused.

In summary, the constructive viewpoint replaces mere satisfiability with witness-bearing realizability. That shift is what makes the section's central metric $\varepsilon(r)$ meaningful: the more structure one proves constructively, the smaller the gap between specification and executable artifact, and the lower the expected counterexample rate in downstream composition and testing.